



UNIVERSITÀ
DEGLI STUDI
FIRENZE

PROJECT REPORT

Student Attendance Tracker

Name: Junaid Munir Janjua
Matricula No: 7131056
Email Id: Junaid.janjua@edu.unifi.it
GitHub: <https://github.com/junaid0000/student-attendance-tracker>

Professor Name: Lorenzo Bettini
Email ID: lorenzo.bettini@edu.unifi.it

Course: Automated Software Testing

LINK TO THE BOOK: <http://leanpub.com/tdd-buildautomation-ci>



TABLE OF CONTENTS

1. Executive Summary.....	4
2. Project Architecture & Tech Stack	4
3. Directory Structure & Organization	6
4. Detailed System Design.....	7
4.1 Models (Entities)	7
4.2 Repositories (Persistence Layer)	7
4.3 Controllers (Logic Layer)	8
4.4 Views (UI Layer)	8
5. Testing Strategy	10
5.1 Unit Testing.....	10
5.2 Integration Testing (IT).....	10
5.3 End-to-End Testing (E2E).....	11
6. Quality Assurance & Analysis Tools	11
7. Build Automation & Dependencies.....	12
8. Build and Execution Instructions	12
9. Conclusion.....	13
10. References	14



UNIVERSITÀ
DEGLI STUDI
FIRENZE

LIST OF FIGURES

Figure 1 Student Management Interface.	9
Figure 2 Attendance Tracking and Reporting Interface.	9
Figure 3 Automated Quality Metrics and CI/CD Status via GitHub Actions.	12



1. Executive Summary

This report presents the design and development of the Student Attendance Tracker, a Java-based desktop application created to efficiently manage student records and monitor daily attendance. The system provides a user-friendly interface that enables administrators and teachers to add, update, and maintain student information while accurately recording attendance data. The project was developed by strictly following Test-Driven Development (TDD) principles, where each functionality was first defined through a failing test case before the actual implementation. This approach ensured that the software remained reliable, well-tested, and less prone to defects throughout the development process. By continuously writing and executing tests, the project achieved improved code quality, easier debugging, and greater confidence in system functionality.

Beyond Test-Driven Development (TDD), the project incorporated contemporary software engineering practices to streamline development activities, strengthen software quality, and support a more efficient and maintainable development process. Build automation helped reduce manual effort and ensured consistent application builds, while continuous integration enabled frequent code integration and automated testing to detect issues early in the development lifecycle. The Student Attendance Tracker demonstrates the practical application of software engineering best practices to develop a maintainable, scalable, and robust attendance management system. The project highlights the importance of automated testing, continuous integration, and disciplined development methodologies in delivering high-quality software solutions.

2. Project Architecture & Tech Stack

The Student Attendance Tracker application is designed using the Model-View-Controller (MVC) architectural pattern, which ensures a clear separation between the user interface, business logic, and data management components. This architecture improves code maintainability, scalability, and overall software quality.



Technologies and Tools Used: This section outlines the programming languages, frameworks, databases, and development tools utilized in the project to ensure efficient development, testing, and deployment.

Programming Language: Java 17

The application is developed using Java 17, which provides improved performance, enhanced security features, and long-term support. Java's platform independence also ensures compatibility across different operating systems.

Database: MongoDB (Java Driver v3.12.7)

MongoDB is used as the primary database for storing student records and attendance information. Its document-oriented structure offers flexibility in managing data, while the MongoDB Java Driver enables efficient communication between the application and the database.

User Interface Framework: Java Swing (Desktop GUI)

Java Swing is utilized to create an interactive and user-friendly desktop graphical user interface. It provides a rich set of built-in components such as buttons, tables, forms, menus, and dialogs, which enable smooth user interaction and efficient management of student and attendance records. The framework ensures consistency in look and feel across different platforms while supporting event-driven programming for responsive application behavior.

Containerization: Docker & Test containers

Docker is used to create consistent and isolated runtime environments for the application, ensuring that it runs the same way across different systems and development stages. Testcontainers supports automated integration testing by spinning up temporary Docker containers during test execution, which provides a clean, controlled, and repeatable testing environment. This approach eliminates environment-related issues and enhances the reliability, consistency, and accuracy of integration



tests, ensuring that test results remain stable across different development and execution environments.

Build Tool: Apache Maven

Apache Maven manages project dependencies, automates the build process, executes tests, and simplifies project configuration. It helps maintain a standardized project structure and ensures consistent builds across different development environments.

3. Directory Structure & Organization

To maintain a structured and professional workflow, the project is organized into dedicated source sets that separate production code from different levels of testing. This organization improves maintainability, simplifies project management, and supports the Test-Driven Development (TDD) approach.

Project Directory Structure:

- **src/main:** Contains the primary application code, including Models, Views, Controllers, Repositories, and business logic.
- **src/test:** Contains Unit Tests that verify the functionality of individual classes and methods in isolation.
- **src/it:** Contains Integration Tests that validate interactions between the application logic and the database.
- **src/e2e:** Contains End-to-End (E2E) Tests that simulate complete user workflows through the graphical user interface.

This structured organization enhances code maintainability, test management, and overall project scalability.



4. Detailed System Design

The Detailed System Design section explains the overall architecture of the system, including its key components, their interactions, and the flow of data between different layers. It provides a clear blueprint of how the application is structured to ensure scalability, maintainability, and efficient functionality.

4.1 Models (Entities)

The Model layer represents the core data structures of the application and defines how information is stored and managed.

Student: Represents a student with a unique roll number and name. The roll number serves as the primary identifier, ensuring that duplicate student records cannot be created.

Attendance Record: Stores attendance information for a student on a specific date. Defensive copying is used for Date objects to maintain data integrity and protect the internal state from unintended modifications.

4.2 Repositories (Persistence Layer)

The application follows the Repository Pattern to separate data access operations from business logic, improving maintainability and flexibility.

- **StudentMongoRepository:** Handles all database operations related to student records.
- **AttendanceMongoRepository:** Manages the storage and retrieval of attendance records within MongoDB.



4.3 Controllers (Logic Layer)

The Controller layer acts as an intermediary between the user interface and the data layer, processing user requests and enforcing business rules.

Student Controller: Manages student registration, updates, and search operations.

Attendance Controller: Handles attendance marking, attendance retrieval, and percentage calculations, including safeguards for empty or missing records.

4.4 Views (UI Layer)

The user interface is implemented through the AttendanceTrackerSwingView, which serves as the primary interaction point between the user and the system. Developed using Java Swing, the interface provides a responsive and user-friendly desktop environment for managing student records and attendance information. The design focuses on simplicity and usability, allowing users to perform operations efficiently without requiring extensive technical knowledge.

- Supports student registration, attendance tracking, and reporting features.
- Includes custom message-handling functionality with auto-resetting timers, providing real-time user feedback while maintaining a clean and organized codebase.



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Student Attendance Tracker

Students Attendance

===== STUDENTS TAB =====

Name:

Roll No:

Add Update Delete

Student List:

0000 - Awais

1234 - junaid

Loaded 2 students from database

Figure 1 Student Management Interface.

Student Attendance Tracker

Students Attendance

===== ATTENDANCE TAB =====

Date:

Students for Attendance:

Awais (0000) ☒ Present ☐ Absent

junaid (1234) ☒ Present ☐ Absent

Mark Attendance

View Attendance: ☒ By Date ☐ By Student

View Attendance Get Summary

Attendance Records:

No attendance records yet.

Figure 2 Attendance Tracking and Reporting Interface.



5. Testing Strategy

The project follows the Test Pyramid approach to achieve a balanced and reliable testing strategy. Different levels of testing are used to validate individual components, component interactions, and complete system functionality. This layered approach helps identify defects early and ensures the overall quality of the application.

5.1 Unit Testing

Unit testing focuses on verifying individual components in isolation using JUnit 4 and Mockito. These tests ensure that each class and method behave as expected without relying on external dependencies.

- Verified the business logic implemented within the Model classes.
- Tested Controller functionality to ensure correct processing of user requests.
- Used Mockito to mock dependencies and verify interactions between Controllers, Views, and Repositories.
- Ensured that View updates and Repository operations were triggered correctly under different scenarios.

5.2 Integration Testing (IT)

Integration testing validates the interaction between the application logic and the MongoDB database. These tests ensure that different components work together correctly when performing real database operations.

- Verified data storage, retrieval, update, and deletion operations in MongoDB.
- Ensured proper communication between Controllers, Repositories, and the database layer.



- Utilized Test containers to create temporary Docker-based MongoDB instances for each test execution.
- Maintained a clean and consistent testing environment by isolating test data from the production database.

5.3 End-to-End Testing (E2E)

End-to-End (E2E) testing was conducted to evaluate the complete system from the user's perspective using black-box testing techniques. These tests were implemented with Assert-Swing to automate graphical user interface (GUI) interactions and verify that all layers of the application work together correctly. Real user actions, such as entering data, clicking buttons, and navigating through the interface, were simulated to test the system under realistic conditions. The tests validated complete workflows, including student registration, attendance marking, and attendance reporting, while ensuring that user actions performed through the interface produced the expected results in the underlying application logic and MongoDB database. By combining Unit, Integration, and End-to-End testing, the project achieved comprehensive test coverage and a high level of software reliability.

6. Quality Assurance & Analysis Tools

To maintain high code quality and ensure software reliability, several industry-standard quality assurance tools were integrated throughout the development process.

- SonarCloud: Performed static code analysis and achieved 0 Code Smells, 0 Vulnerabilities, and 0 Bugs.
- JaCoCo: Measured test coverage and achieved 100% coverage of the core business logic.
- GitHub Actions: Automated the Continuous Integration (CI) pipeline by executing tests and quality checks on every push and pull request.



UNIVERSITÀ
DEGLI STUDI
FIRENZE

🔗 Student Attendance Tracker



Figure 3 Automated Quality Metrics and CI/CD Status via GitHub Actions.

7. Build Automation & Dependencies

Testing Frameworks: Unit 4, AssertJ, Mockito, AssertJ-Swing, and Awaitility were used to implement unit, integration, and end-to-end testing.

Database Management: MongoDB Java Driver and Testcontainers were utilized for database connectivity and isolated integration testing.

Maven Plugins:

Surefire: Executed unit and integration tests.

Failsafe: Managed the execution of end-to-end (E2E) tests.

PIT: Performed mutation testing to assess the effectiveness and robustness of the test suite.

8. Build and Execution Instructions

To compile, test, and execute the application locally, ensure that Java 17, Apache Maven, and Docker Desktop (running) are installed on the system.



1. Clean and Build (No Tests)
 - mvn clean package -DskipTests
2. Run Standard Unit Tests Only
 - mvn clean test
3. Run ALL Tests (Unit + Integration + E2E)
 - mvn clean verify
4. Run ALL Tests + Generate Coverage Report (JaCoCo)
 - mvn clean verify -Pjacoco
5. Run Mutation Testing (PIT)
 - mvn clean verify -Ppit
6. Run Tests WITHOUT Docker
 - mvn clean verify -Pskip-testcontainers

9. Conclusion

The Student Attendance Tracker project successfully demonstrates the practical application of modern software engineering principles, extending beyond basic software development to a structured and quality-driven development process. By combining Test-Driven Development (TDD), the MVC architectural pattern, and automated quality assurance practices, the project delivers a robust, maintainable, and scalable application.

Furthermore, the integration of Docker and Testcontainers ensures portability, consistency, and reproducibility across different development environments. The achievement of high-test coverage and successful compliance with SonarCloud quality standards provide strong evidence of the application's reliability and code quality.

This project serves as a practical example of how automated testing, continuous integration, and modern development practices can enhance software quality, reduce technical debt, and support long-term system maintainability.



UNIVERSITÀ
DEGLI STUDI
FIRENZE

10. References

- [1]. Book Example Repository: <https://github.com/torenzoBettini/tdd-buildautomation-ci-book-examples>
- [2]. LINK TO THE BOOK: <http://leanpub.com/tdd-buildautomation-ci>
- [3]. GitHub: github.com/junaaid0000/student-attendance-tracker